

# ServMate

## Complete Technical Documentation

AI-Assisted Hyperlocal Service Marketplace

Generated: 2/9/2026

---

# 01-project-overview

## 01 - Project Overview

**Project Name:** ServMate  
**Project Category:** AI-Assisted Hyperlocal Service Marketplace  
**Architecture Style:** Modular Monolith  
**Current Phase:** Phase 1 - Project Initialization & Foundation Setup

### 1. Problem Statement

Urban and suburban customers frequently struggle to find reliable, verified, and prompt local service professionals (electricians, plumbers, appliance repair technicians, cleaners, etc.). Existing platforms often lack:

- Transparent verification of service providers.
- Real-time tracking and location/availability-based matching.
- Fraud prevention during service execution (e.g., billing without service or unauthorized technician swap).
- Proof of work before/after completion.

### 2. ServMate Solution

ServMate bridges this gap by introducing an AI-assisted hyperlocal marketplace that connects verified service providers with local customers. Features include:

1. **Verified Provider Profiles:** Admin verification badge system.
2. **Availability & Proximity Matching:** Smart provider recommendations based on distance, experience, ratings, and real-time schedule.
3. **OTP-Based Verification:** Secure service initiation using customer-generated OTPs.
4. **Before/After Photo Proof:** Provider uploads visual evidence of work completion stored securely.
5. **AI Assistance:** Automated classification of service requests and customer issues using Gemini/OpenAI APIs.

### 3. Scope and Implementation Status

#### Implemented (Phase 1)

- [x] Project Scaffolding (Monorepo-style structure with ``/backend``, ``/frontend``, ``/docs``).
- [x] Backend Express Server with security middleware (``helmet``, ``cors``), Zod validation framework, and centralized error handler.
- [x] PostgreSQL database integration via Prisma ORM.
- [x] React + Vite + Tailwind CSS frontend shell.
- [x] Environment configuration setup (`` .env.example `` templates).
- [x] Technical PDF Documentation Generator.

#### Planned (Future Phases)

- [ ] User & Provider Authentication & Authorization (``STATUS: PLANNED``)
- [ ] Service Profiles & Verification Engine (``STATUS: PLANNED``)
- [ ] Booking Workflow & OTP Verification (``STATUS: PLANNED``)
- [ ] Smart Distance & Rating Matching Algorithm (``STATUS: PLANNED``)
- [ ] Real-time Socket.io Chat (``STATUS: PLANNED``)
- [ ] Redis Caching Layer (``STATUS: PLANNED``)
- [ ] Cloudinary Proof Upload Integration (``STATUS: PLANNED``)
- [ ] AI Service Classification (``STATUS: PLANNED``)

# 02-system-architecture

## 02 - System Architecture

**Current Phase Status:** Implemented (Phase 1 Foundation Setup)

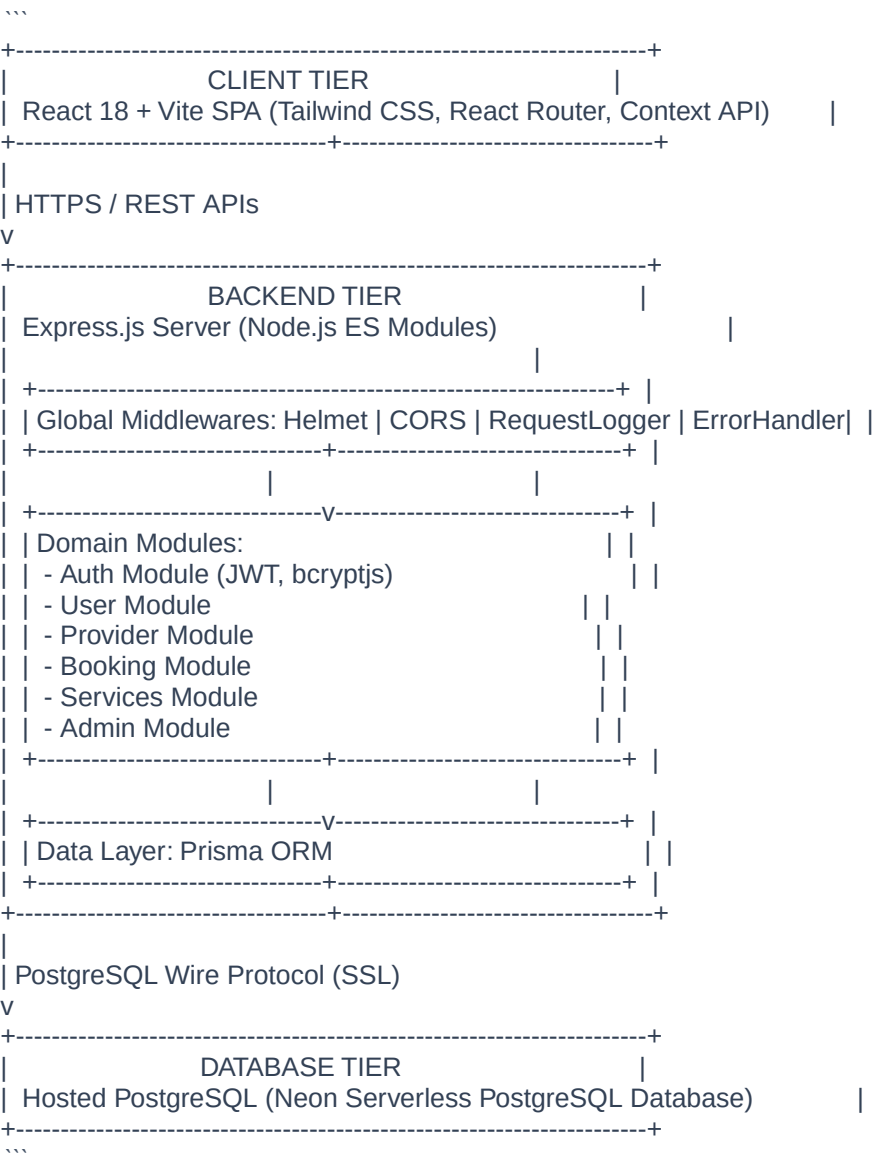
### 1. Architectural Strategy: Modular Monolith

ServMate avoids microservice over-engineering by using a **Modular Monolith** pattern inside a single Node.js Express backend repository (`/backend`).

#### Why Modular Monolith?

- High Cohesion, Low Coupling:** Each domain feature (Auth, Users, Providers, Bookings, Services, Reviews) lives in its dedicated module folder with its own routes, controller, service, and validation logic.
- Simplified Deployment:** Single deployment unit to platforms like Render, avoiding multi-container orchestration overhead.
- Easy Transition:** If any module (e.g., Notification or AI) experiences extreme load, its clean internal boundaries make it easy to break out into a microservice in the future.

### 2. High-Level System Architecture Diagram



### 3. Directory Layout Standard

```
...
backend/src/
% % % config/      # App & DB configuration
% % % middlewares/  # Express global & route middlewares
% % % utils/        # Helper utilities & custom AppError
% % % modules/      # Domain Business Modules
% % % auth/         # Authentication endpoints & business logic
% % % users/        # User profile operations
% % % providers/    # Provider verification & listings
% % % bookings/     # Booking & OTP execution workflow
% % % services/     # Service catalog management
% % % admin/        # Admin oversight endpoints
...
```

# 03-tech-stack

## 03 - Tech Stack

**Current Phase Status:** Implemented (Phase 1 Foundation Setup)

### 1. Core Stack Overview

| Layer              | Technology                              | Reason Selected                                                                       |
|--------------------|-----------------------------------------|---------------------------------------------------------------------------------------|
| Frontend Framework | React 18 + Vite                         | Lightning-fast build speeds, component-driven UI architecture.                        |
| Styling            | Tailwind CSS                            | Utility-first CSS for rapid, modern, responsive UI design without bulky CSS files.    |
| Routing & State    | React Router DOM v6 & React Context API | Clean client-side routing and native state management without Redux overhead.         |
| HTTP Client        | Axios                                   | Declarative HTTP client with interceptors for JWT token injection and error handling. |
| Backend Runtime    | Node.js (v18+) with ES Modules          | Standard JavaScript runtime enabling modern <code>import/export</code> syntax.        |
| Web Server         | Express.js                              | Industry-standard minimal, flexible web framework for RESTful APIs.                   |
| ORM                | Prisma ORM                              | Type-safe database client, auto-generated migrations, intuitive schema definition.    |
| Data Validation    | Zod                                     | Schema-first TypeScript/JavaScript validation for API body, query, and params.        |
| Database           | Hosted PostgreSQL (Neon)                | Reliable serverless relational database with ACID compliance and SSL encryption.      |
| Security           | Helmet & CORS                           | Helmet sets HTTP security headers; CORS manages cross-origin resource access.         |
| Authentication     | JWT & bcryptjs                          | Stateless token authentication and salted password hashing.                           |

### 2. Planned Technologies (STATUS: PLANNED)

- Redis:** Caching layer for provider availability and rate limiting.
- Socket.io:** WebSockets for real-time customer-provider chat and booking updates.
- Cloudinary:** Cloud image storage for provider ID verification and before/after service proof.
- Gemini / OpenAI API:** AI service request classification and issue tagging.
- Nodemailer:** Email notifications for booking confirmation and status changes.
- Jest & Supertest:** Automated unit and integration testing suite.

# 04-database-design

## 04 - Database Design

**\*\*Current Phase Status\*\*:** Implemented (Phase 1 Initial Prisma Schema Setup)

---

### 1. Relational Database Selection: PostgreSQL (Neon)

ServMate requires strict ACID compliance for financial transactions, bookings, user profiles, and verification states. PostgreSQL hosted on Neon provides serverless scaling, point-in-time recovery, and reliable cloud connections.

---

### 2. Core Entities & Prisma Schema (Phase 1 Baseline)

#### User & Verification Entities

```
``prisma
enum Role {
  CUSTOMER
  PROVIDER
  ADMIN
}

enum ServiceStatus {
  PENDING
  CONFIRMED
  IN_PROGRESS
  COMPLETED
  CANCELLED
}

model User {
  id      String  @id @default(uuid())
  email   String  @unique
  password String
  fullName String
  phone   String?
  role    Role    @default(CUSTOMER)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
...

```

---

### 3. Planned Models (`STATUS: PLANNED`)

- ``ProviderProfile``: Provider specialization, experience years, hourly rate, verification status, geolocation (latitude, longitude).
- ``ServiceCategory``: Catalog of service categories (Electrician, Plumber, Appliance, Cleaning).
- ``Booking``: Customer service bookings, scheduled time, OTP start verification code, status, price.
- ``ServiceProof``: Before and after service completion images stored on Cloudinary.
- ``Review``: Star ratings (1-5) and feedback left by customers for service providers.

# 05-api-documentation

## 05 - API Documentation

**\*\*Current Phase Status\*\*:** Implemented (Phase 1 Baseline Endpoints)

---

### 1. Implemented Endpoints (Phase 1)

#### ``GET /api/v1/health``

- **\*\*Purpose\*\*:** System health check endpoint for uptime monitoring and deployment verification.
- **\*\*Authentication\*\*:** None Required
- **\*\*Authorization\*\*:** Public
- **\*\*Request Body\*\*:** None
- **\*\*Query Parameters\*\*:** None
- **\*\*Success Response\*\*** (``200 OK``):

```
```json
{
  "status": "success",
  "message": "ServMate API is healthy and operational",
  "timestamp": "2026-08-20T12:00:00.000Z",
  "environment": "development"
}
```
```

---

### 2. Planned API Specification (``STATUS: PLANNED``)

#### Auth Module (``STATUS: PLANNED``)

- ``POST /api/v1/auth/register``: Register new Customer/Provider
- ``POST /api/v1/auth/login``: User authentication & JWT issue

#### Provider Module (``STATUS: PLANNED``)

- ``POST /api/v1/providers/profile``: Create/update service provider profile
- ``GET /api/v1/providers/search``: Smart search and proximity matching

#### Booking Module (``STATUS: PLANNED``)

- ``POST /api/v1/bookings``: Create booking request
- ``POST /api/v1/bookings/:id/verify-otp``: Service start OTP verification

# 06-authentication

## Chapter 6: Authentication & Authorization System

### Overview

ServMate implements a stateless, token-based authentication system using **JSON Web Tokens (JWT)**, **bcryptjs password hashing**, and **Zod schema validation**.

### Architecture & Flows

#### 1. User Registration Flow (`POST /api/v1/auth/register`)

- Public Roles Allowed**: `CUSTOMER`` and `PROVIDER``.
- Public Admin Block**: Registration requests specifying `role: "ADMIN"` are rejected (400 Bad Request).
- Validation**: Zod schema validates email syntax, phone format, and password length (≥ 6 characters).
- Password Security**: Passwords are salted and hashed using `bcryptjs`` (10 rounds) before database storage.
- Response**: Returns JWT token and user object. Password hash is explicitly omitted (`select`` clause).

#### 2. User Login Flow (`POST /api/v1/auth/login`)

- Credentials Check**: Verifies email existence and compares password hash using `bcrypt.compare``.
- Generic Errors**: Returns standard `401 Unauthorized`` ("Invalid email address or password") to prevent user enumeration attacks.
- Status Check**: Deactivated users (`isActive: false``) are rejected with `403 Forbidden``.
- JWT Payload**: Claims include `id`` and `role``. Expiration default: `7d``.

#### 3. Role-Based Authorization Middleware (`authorizeRoles``)

- Restricts API routes based on user role (`CUSTOMER``, `PROVIDER``, `ADMIN``).
- Unauthenticated requests  $\rightarrow$  `401 Unauthorized``.
- Unauthorized roles  $\rightarrow$  `403 Forbidden``.

### Logout Design Decision

JWT is stateless. Logout is executed on the client-side by purging the token from `localStorage`` and clearing the `Authorization`` header. Server-side token blacklisting is marked as `STATUS: PLANNED (PHASE 2)``.



# 07-provider-system

## 07 — Provider System & Interactive Map Location Picker

### 1. Overview

The ServMate Provider System empowers verified local professionals to showcase their trade capabilities, experience, hourly rates, service categories, availability schedules, and precise service location.

A crucial component of hyperlocal service delivery is geographical proximity. Providers must specify where they operate so the ServMate Smart Matching Engine can calculate distance, compute proximity scores, and match them with nearby customers.

---

### 2. Interactive Map Location Picker

#### Why Leaflet & OpenStreetMap Were Selected

1. **Open Source & Cost-Effective**: Eliminates dependency on proprietary, paid mapping SDKs (e.g. Google Maps Platform) and avoids credit card requirements, billing surprises, or API key restrictions.
2. **Lightweight & High Performance**: Leaflet is a compact JavaScript mapping library (~42 KB gzip) that performs exceptionally well across mobile and desktop devices.
3. **No Secret Keys in Frontend**: Tile requests are made via standard HTTPS OpenStreetMap endpoints (`https://tile.openstreetmap.org/{z}/{x}/{y}.png`), eliminating the risk of client-side credential exposure.
4. **Reliable Community Data**: OpenStreetMap provides global, crowd-verified geographic map coverage with regular updates.

---

### 3. Map Coordinates & User Interaction

The `LocationPicker` component (`frontend/src/components/LocationPicker.jsx`) provides an intuitive visual interface:

...

Provider opens Profile

!“

Map renders centered on saved location (or Mumbai default [19.0760, 72.8777])

!“

Provider can:

- a) Click anywhere on the map to place a pin
- b) Drag the existing pin to fine-tune exact location
- c) Click "Use My Current Location" (Browser Geolocation API)
- d) Search area or landmark via explicit search bar

!“

Marker coordinates captured: { latitude, longitude }

!“

Read-only transparency preview displayed below map

!“

Form submits latitude & longitude to backend API

...

#### How Coordinates Reach the Backend & Database

1. **Capture**: React state in `LocationPicker` captures `latLng.lat` and `latLng.lng` (rounded to 6 decimal places).
2. **Transmission**: When the provider saves their profile, the frontend issues an authenticated `POST /api/v1/providers/profile` or `PUT /api/v1/providers/profile/me` payload with JSON fields:

```json`

`{`

`"bio": "Certified residential electrician",`

`"experienceYears": 7,`

`"hourlyRate": 450,`

`"city": "Mumbai",`

`"state": "Maharashtra",`

`"latitude": 19.0760,`

`"longitude": 72.8777`

}  
...

3. **Zod Validation**: The backend validates coordinate ranges:

- Latitude: `-90 <= latitude <= 90`
- Longitude: `-180 <= longitude <= 180`

4. **PostgreSQL Persistence**: Stored directly in the `Provider` table under `latitude Float?` and `longitude Float?`.

---

## 4. Browser Geolocation Flow

When the provider clicks **"Use My Current Location"**:

1. Invokes the standard W3C `navigator.geolocation.getCurrentPosition()`.
2. The browser requests one-time permission from the user.
3. If granted, coordinates `{ coords: { latitude, longitude } }` are retrieved with high accuracy.
4. The Leaflet map smoothly animates (`map.setView([lat, lon], 15)`), places the marker, and updates the form state.
5. If denied, timed out, or unavailable, clear localized error messages guide the user to click the map manually without breaking application state.

---

## 5. How Coordinates Are Used in Haversine Smart Matching

ServMate's Smart Matching algorithm uses the **Haversine formula** to calculate great-circle distance between the customer's requested service location and the provider's coordinates:

$$\text{distance} = 2 R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta \text{lat}}{2}\right) + \cos(\text{lat}_1)\cos(\text{lat}_2)\sin^2\left(\frac{\Delta \text{lon}}{2}\right)}\right)$$

- **Weight in Matching Score**: 20%
- **Decay Curve**: Linear decay from 1.0 (0 km) down to 0.0 at 50 km (`Math.max(0, 1 - distKm / 50)`).
- **Fallback**: If coordinates are missing on either side, the system gracefully falls back to city name matching (10% score).

---

## 6. OpenStreetMap Attribution & Nominatim Usage Policy

- **Tile Attribution**: Under the OpenStreetMap copyright policy, the map displays attribution:

© OpenStreetMap contributors

- **Nominatim Geocoding Policy**:
  - Maximum 1 request per second.
  - No autocomplete or search-as-you-type querying.
  - Geocoding requests are only performed on explicit user button clicks.
  - Proxied via the ServMate backend with a compliant `User-Agent: ServMate-AI/1.0 (contact: support@servmate.ai)` header.
- **Non-blocking**: If Nominatim is unavailable, coordinate selection and profile saving continue unaffected.

# 08-booking-system

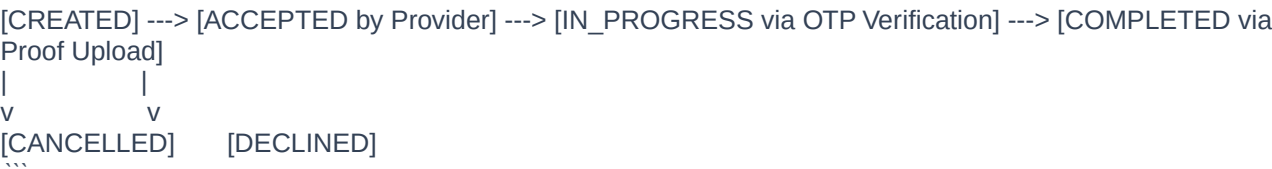
## 08 - Booking System & Availability Workflow

**\*\*STATUS: PLANNED\*\*** (Future Phase Feature)

---

### 1. Planned Booking State Machine

...



### Planned Features

- Availability calendar checking.
- OTP code generated for customer upon provider arrival.
- Service execution unlock only when correct OTP is verified by provider app.

# 09-realtime-system

## ServMate — Real-Time System (Phase 3)

### Overview

Phase 3 adds real-time customer-provider communication via **Socket.io**, backed by **PostgreSQL** for persistence and **Upstash Redis** for ephemeral state (OTPs, rate limits).

### Socket.io Architecture

#### Server Setup

- Socket.io is attached to the Node.js **HTTP server** (not Express directly)
- This is required so that the same TCP port handles both HTTP (REST) and WebSocket (Socket.io) traffic
- The `io` instance is made available throughout the application via `app.set('io', io)`

```
HTTP Server (port 5000)
% % % Express App (HTTP/REST)
% % % Socket.io Server (WebSocket/polling)
```

#### JWT Authentication Middleware

Every socket connection is authenticated **before** it is allowed to connect:

```
``js
io.use(async (socket, next) => {
  const token = socket.handshake.auth?.token;
  if (!token) return next(new Error('Authentication required'));
  const decoded = jwt.verify(token, JWT_SECRET);
  socket.userId = decoded.id;
  socket.userRole = decoded.role;
  next();
});
```

Unauthenticated or invalid-token connections receive a `connect_error` event and are dropped.

#### Room Strategy

| Room                                 | Members                                          | Purpose                                  |
|--------------------------------------|--------------------------------------------------|------------------------------------------|
| <code>`user:{userId}`</code>         | Auto-joined on connect                           | Personal notifications (booking updates) |
| <code>`conv:{conversationId}`</code> | Joined via <code>join_conversation`</code> event | Real-time chat messages                  |

Joining a conversation room is **authorization-checked** — only confirmed participants can join.

### Socket Events

#### Client !' Server

| Event                            | Payload                                  | Description                                  |
|----------------------------------|------------------------------------------|----------------------------------------------|
| <code>`join_conversation`</code> | <code>{ conversationId }</code>          | Join a conversation room (participant-only)  |
| <code>`send_message`</code>      | <code>{ conversationId, content }</code> | Send a message (persisted to DB + broadcast) |
| <code>`mark_read`</code>         | <code>{ conversationId }</code>          | Mark messages in conversation as read        |

#### Server !' Client

| Event                              | Payload                                 | Description                   |
|------------------------------------|-----------------------------------------|-------------------------------|
| <code>`new_message`</code>         | <code>{ message }</code>                | New message in a conversation |
| <code>`messages_read`</code>       | <code>{ conversationId, readBy }</code> | Messages marked as read       |
| <code>`booking_update`</code>      | <code>{ bookingId, status }</code>      | Booking status changed        |
| <code>`joined_conversation`</code> | <code>{ conversationId }</code>         | Confirmation of room join     |

| `error` | `{ message }` | Socket-level error |

---

## Message Persistence

PostgreSQL is the **source of truth** for all messages. Socket.io only handles delivery.

Flow:

---

```
Client sends 'send_message'
!' Server validates participant
!' Server saves to PostgreSQL (prisma.message.create)
!' Server broadcasts 'new_message' to room
!' All clients in room receive message
---
```

If the socket is not connected, clients fall back to the REST `POST /conversations/:id/messages` endpoint, which also emits to the socket room if available.

---

## Conversation Authorization

- A conversation is created by a **customer** linked to a specific **booking**
- Only the `customerId` and `providerId` from that conversation record can access messages
- Any attempt by a third party returns `404` (not `403`) to avoid leaking conversation existence

---

## Real-Time Booking Events

When booking status changes (accept, reject, cancel, OTP verify, complete), the backend emits `booking\_update` to both `user:{customerId}` and `user:{providerId}` rooms.

```js

```
io.to(`user:${customerId}`).emit('booking_update', { bookingId, status });
io.to(`user:${providerId}`).emit('booking_update', { bookingId, status });
---
```

PostgreSQL is updated **first**, then the socket event is emitted.

---

## Reconnection

Socket.io client is configured with:

- `reconnection: true`
- `reconnectionAttempts: 5`
- `reconnectionDelay: 1000ms`

On reconnect, the client re-joins conversation rooms via `join\_conversation`.

---

## Frontend Implementation

- `frontend/src/api/socket.js` — singleton socket client with lazy connection
- `frontend/src/pages/Messages.jsx` — shared chat UI for customer and provider
- Routes: `/customer/messages` and `/provider/messages`

# 10-service-verification

## ServMate — Service Verification System (Phase 3)

### Purpose

The service-start OTP verification ensures that the \*\*customer and provider are physically together\*\* before service begins, preventing fraudulent service claims.

### OTP Flow

```
...
Booking CONFIRMED
!"
Customer opens booking !' taps "Generate Service OTP"
!"
Backend generates 6-digit OTP via crypto.randomInt()
!"
OTP stored in Redis with 5-minute TTL
!"
Customer shows OTP on screen to provider (verbal share)
!"
Provider enters 6 digits in their app !' hits "Verify OTP"
!"
Backend verifies:
  • OTP exists in Redis (not expired)
  • OTP matches stored value
  • Attempt count < 5
!"
OTP deleted from Redis (one-time use)
!"
Booking status !' SERVICE_STARTED
!"
Real-time socket event sent to both parties
...
```

### Security Details

| Property          | Implementation                                                |
|-------------------|---------------------------------------------------------------|
| Generation        | `crypto.randomInt(100000, 999999)` — cryptographically random |
| Length            | 6 digits                                                      |
| Storage           | Redis only (never PostgreSQL)                                 |
| Expiry            | 5-minute TTL via Redis `EX` parameter                         |
| Max attempts      | 5 per OTP — exceeded !' OTP deleted                           |
| Rate limit        | Max 3 OTP generations per customer per minute                 |
| One-time use      | Deleted immediately upon successful verification              |
| Role enforcement  | Only CUSTOMER can generate; only PROVIDER can verify          |
| Booking ownership | Provider can only verify OTP for bookings assigned to them    |

### Redis Key Schema

```
...
otp:{bookingId}    !' JSON { otp: "123456", attempts: 0 }
TTL: 300 seconds

otp_gen:{userId}   !' integer (count)
TTL: 60 seconds (rate limit window)
...
```

## Booking Status Lifecycle (Extended)

```
...
PENDING
!" (provider accepts)
CONFIRMED
!" (customer generates OTP !' provider verifies)
SERVICE_STARTED
!" (provider marks complete)
COMPLETED

Alternative paths:
PENDING !' REJECTED    (provider rejects)
PENDING !' CANCELLED   (customer cancels)
CONFIRMED !' CANCELLED (customer cancels)
...
```

All invalid transitions are blocked on the backend with HTTP 400.

---

## API Endpoints

### Generate OTP

```
...
POST /api/v1/bookings/:id/otp/generate
Authorization: Bearer {customerToken}
Role: CUSTOMER

Response 200:
{
  "status": "success",
  "data": {
    "otp": "482951",
    "expiresInSeconds": 300,
    "note": "& p Dev/Demo mode — display only on customer device"
  }
}
...
```

### Verify OTP

```
...
POST /api/v1/bookings/:id/otp/verify
Authorization: Bearer {providerToken}
Role: PROVIDER

Body: { "otp": "482951" }

Response 200:
{
  "status": "success",
  "data": { "booking": { "id": "...", "status": "SERVICE_STARTED" } }
}
...
```

---

## What Happens If Redis is Unavailable

- `getRedis()` throws a structured error with `statusCode: 503`
- The error propagates through the Express error handler
- Client receives `503 Service Unavailable` with a descriptive message
- Application does NOT silently pretend Redis is working
- All other endpoints (REST, auth, bookings) continue to function normally

---

## Development / Demo Note

In development, the OTP is returned to the customer in the API response for convenience. In production, this should only be displayed on the customer's device screen and never transmitted via network to anyone other

than the customer.



# 11-ai-system

## 11 - AI System & Intelligent Assistance

**\*\*STATUS: PLANNED\*\*** (Future Phase Feature)

---

### 1. Planned Overview

Integration with Google Gemini API / OpenAI API for:

- Automatic categorization of customer problem descriptions into service tags.
- Urgency scoring and initial diagnostic troubleshooting steps for customers.

# 12-smart-matching

## 12 - Smart Provider Matching Algorithm

**\*\*STATUS: PLANNED\*\*** (Future Phase Feature)

---

### 1. Planned Scoring Formula

Provider rank score will be computed using a weighted multi-factor algorithm:

$$\text{Score} = (W_d \cdot \text{DistanceScore}) + (W_r \cdot \text{Rating}) + (W_e \cdot \text{Experience}) + (W_a \cdot \text{Availability})$$

Where distance is computed using the Haversine formula based on provider and customer coordinates.

# 13-redis

## ServMate — Redis Integration (Phase 3)

### Why Redis?

PostgreSQL is the **source of truth** for all persistent data in ServMate. Redis is used exclusively for **ephemeral, time-sensitive data** where:

1. Automatic expiration (TTL) is required
2. Data must not persist beyond its validity window
3. Fast read/write latency is important
4. Persistence is NOT desired (OTPs should disappear after use)

---

### Redis Provider: Upstash

ServMate uses **Upstash Redis** — a serverless, REST-based Redis service that requires no TCP connection or local installation.

- **Client library:** `@upstash/redis` (npm)
- **Connection:** HTTPS REST API (no TCP socket)
- **Credentials:** `UPSTASH_REDIS_REST_URL` + `UPSTASH_REDIS_REST_TOKEN`
- **Free tier:** 10,000 requests/day

---

### Key Schema

| Key Pattern                     | Value                               | TTL          | Purpose                       |
|---------------------------------|-------------------------------------|--------------|-------------------------------|
| <code>`otp:{bookingId}`</code>  | JSON { otp: "123456", attempts: 0 } | 300s (5 min) | Service start OTP             |
| <code>`otp_gen:{userId}`</code> | integer (count)                     | 60s          | OTP generation rate limit     |
| <code>`test:{any}`</code>       | any                                 | varies       | Integration test verification |

---

### OTP Storage

```
``javascript
// Store
await redis.set(`otp:${bookingId}`, JSON.stringify({ otp, attempts: 0 }), { ex: 300 });

// Retrieve
const stored = await redis.get(`otp:${bookingId}`);
const data = JSON.parse(stored); // { otp: "482951", attempts: 1 }

// Delete (after use)
await redis.del(`otp:${bookingId}`);
``
```

---

### Rate Limiting

#### OTP Generation Rate Limit

Prevents a customer from flooding OTP generation requests.

```
``javascript
const key = `otp_gen:${userId}`;
const count = await redis.incr(key);
if (count === 1) await redis.expire(key, 60); // 60-second window
if (count > 3) throw new AppError('Too many OTP requests', 429);
``
```

| Parameter     | Value                           |
|---------------|---------------------------------|
| Window        | 60 seconds                      |
| Max requests  | 3 per window                    |
| Key structure | <code>`otp_gen:{userId}`</code> |

| HTTP response on breach | 429 Too Many Requests |

---

## Failure Handling

If Redis is unavailable (network issue, quota exceeded):

1. `getRedis()` throws a structured error: `{ message: 'Redis unavailable', statusCode: 503 }`
2. Error propagates through Express error handler
3. Client receives `503 Service Unavailable`
4. All non-Redis endpoints (auth, bookings, providers, etc.) continue normally
5. OTP operations are **blocked** — they do not silently succeed

*```javascript*

```
export const getRedis = () => {
  if (!redis) {
    const err = new Error('Redis unavailable — cannot complete this operation');
    err.statusCode = 503;
    throw err;
  }
  return redis;
};
```
```

---

## PostgreSQL vs Redis — When to Use Which

| Data | Storage | Reason |

|-----|-----|-----|

| User accounts | PostgreSQL | Permanent, needs relationships |  
| Bookings | PostgreSQL | Permanent business records |  
| Messages | PostgreSQL | Permanent conversation history |  
| Reviews | PostgreSQL | Permanent, affects provider rating |  
| Service proof | PostgreSQL (metadata) | Permanent evidence |  
| OTP | **Redis** | Must expire; one-time use |  
| Rate limit counters | **Redis** | Time-windowed, ephemeral |  
| Session data | Neither (JWT stateless) | Stateless auth |

---

## Interview Q&A

**Q:** Why not store OTPs in PostgreSQL?

A: PostgreSQL doesn't support automatic row expiration. We would need a scheduled cleanup job. Redis TTL handles expiration atomically and automatically, which is the correct tool for ephemeral data.

**Q:** What if Redis goes down?

A: OTP operations fail with 503. All other features (auth, booking management, messaging, reviews) continue working because they don't depend on Redis.

**Q:** Why Upstash instead of local Redis?

A: Upstash is serverless, requires no local install, works identically in development and production, and has a free tier sufficient for this project.

# 14-testing

## 14 - Testing Strategy

**\*\*Current Phase Status\*\***: Implemented (Phase 1 Health Endpoint Verification & Build Checks)

---

### 1. Implemented Phase 1 Verification

- Express API server startup & route registration check.
- Health check HTTP endpoint test (`GET /api/v1/health`).
- Frontend Vite build and Tailwind CSS compilation check.
- Automated PDF compilation test script (`npm run docs:pdf`).

---

### 2. Planned Testing Framework (`STATUS: PLANNED`)

- **\*\*Unit Testing\*\***: Jest for utility functions and algorithm scoring logic.
- **\*\*Integration Testing\*\***: Supertest for REST API route integration tests with an in-memory/test PostgreSQL database instance.

# 15-security

## 15 - Security Standards & Practices

**\*\*Current Phase Status\*\***: Implemented (Phase 1 Baseline Security Setup)

---

### 1. Implemented Baseline Security

1. **\*\*Helmet Middleware\*\***: Secures Express apps by setting HTTP headers (`X-Content-Type-Options`, `X-Frame-Options`, `X-XSS-Protection`, etc.).
2. **\*\*CORS Configuration\*\***: Restricts cross-origin requests to configured frontend domains.
3. **\*\*Zod Validation\*\***: Input sanitization and schema enforcement on HTTP requests to prevent injection vulnerabilities.
4. **\*\*Environment Secrets Protection\*\***: Complete exclusion of secrets (`.env`) from version control via strict `.gitignore`.

---

### 2. Planned Security Implementations (`STATUS: PLANNED`)

- Salted Password Hashing via `bcryptjs` (salt rounds  $\geq 10$ ).
- Rate limiting middleware on sensitive auth routes (`express-rate-limit` / Redis).
- HTTPS enforcement on production deployment (Render/Vercel).

# 16-deployment

## 16 - Deployment Architecture

**\*\*Current Phase Status\*\***: Implemented (Phase 1 Build & Environment Scaffolding)

---

### 1. Planned Deployment Infrastructure (`STATUS: PLANNED`)

- **\*\*Frontend\*\***: Vercel (Static Web App deployment for React SPA built with Vite).
- **\*\*Backend\*\***: Render (Node.js web service running Express API).
- **\*\*Database\*\***: Hosted PostgreSQL (Neon serverless PostgreSQL database with SSL connection pooling).

# 17-environment-variables

## 17 - Environment Variables Guide

**\*\*Current Phase Status\*\*:** Implemented (Phase 1 Baseline Templates)

---

### 1. Backend Environment Variables (`backend/.env.example`)

``env

#### Server Configuration

PORT=5000

NODE\_ENV=development

#### Database Connection (Neon PostgreSQL)

DATABASE\_URL="postgresql://user:password@ep-xyz.region.aws.neon.tech/servmate?sslmode=require"

#### JWT Configuration (Phase 2)

JWT\_SECRET=your\_super\_secret\_jwt\_key\_min\_32\_characters

JWT\_EXPIRES\_IN=7d

#### CORS Configuration

CLIENT\_URL=http://localhost:5173

``

---

### 2. Frontend Environment Variables (`frontend/.env.example`)

``env

#### API Base URL

VITE\_API\_BASE\_URL=http://localhost:5000/api/v1

``



# 18-external-integrations

## 18 - External Integrations

**\*\*Current Phase Status\*\***: Phase 1 Database Integration Setup

---

### 1. Implemented Integrations

#### Neon Hosted PostgreSQL Database

- **\*\*Official Website\*\***: <https://neon.tech>
- **\*\*Purpose\*\***: Serverless PostgreSQL database hosting for production and development data persistence.
- **\*\*Account Requirements\*\***: Free account registered at [neon.tech](https://neon.tech).
- **\*\*Credentials / API Key\*\***: Connection String (`DATABASE\_URL`).
- **\*\*Where Stored\*\***: `backend/.env` file under key `DATABASE\_URL`.
- **\*\*Free Tier Available\*\***: Yes (0.5 GiB storage, free compute units, branch database environments).
- **\*\*Security Considerations\*\***: SSL mode enforced (`?sslmode=require`). Credentials are never hardcoded or checked into version control.

---

### 2. Planned Integrations (`STATUS: PLANNED`)

- **\*\*Cloudinary\*\***: Media storage for provider ID verification documents and service proof photos.
- **\*\*Google Gemini API / OpenAI\*\***: Natural language service request classification and issue diagnostic tagging.
- **\*\*Nodemailer / SMTP Service\*\***: Transactional email dispatch for booking confirmations and OTP verification.

# 19-interview-preparation

## ServMate — Interview Preparation (Phase 1, 2 & 3)

### Phase 1 — Authentication & Authorization

**\*\*Q1: Why JWT over sessions?\*\***

JWT is stateless — no server-side session store needed. Each token contains the user ID and role, verified by the server with a secret key. This scales horizontally without sticky sessions.

**\*\*Q2: How does bcrypt hashing work?\*\***

bcrypt is a one-way adaptive hashing function. The salt is generated and embedded in the hash. Even if two users have the same password, their hashes differ due to unique salts. The `cost factor` (rounds) makes brute-force exponentially harder.

**\*\*Q3: How does role-based authorization work?\*\***

The JWT contains the user's `role`. The `authorizeRoles` middleware checks `req.user.role` against allowed roles for each route. If the role doesn't match, a 403 is returned before the controller runs.

---

### Phase 2 — Provider Marketplace & Booking

**\*\*Q4: How does double-booking prevention work?\*\***

Two layers: (1) An `@@unique([providerId, bookingDate, startTime, endTime])` schema constraint at the PostgreSQL level. (2) A `prisma.\$transaction` check that queries for existing conflicting bookings before creating a new one. Both layers together prevent any race condition.

**\*\*Q5: How does the Haversine matching algorithm work?\*\***

Haversine calculates the great-circle distance between two GPS coordinates. It uses the law of haversines with Earth's radius (~6371 km) to compute distance without Euclidean approximation errors. The matching service uses it to score proximity between customer and provider.

**\*\*Q6: What are the 6 factors in the smart matching score?\*\***

Service match (30%), distance (20%), availability (20%), average rating (15%), experience (10%), hourly rate below average (5%). These weights are applied to give a transparent score between 0-100.

---

### Phase 3 — Real-Time, Redis, OTP, Reviews

**\*\*Q7: Why Socket.io instead of raw WebSockets?\*\***

Socket.io adds: (1) Automatic fallback to long-polling if WebSocket fails. (2) Built-in reconnection. (3) Room/namespace management. (4) Event-based API vs raw binary frames. For a real-time chat application, these features save significant development time.

**\*\*Q8: How does Socket.io authentication work?\*\***

The client sends the JWT in `socket.handshake.auth.token`. The server's `io.use()` middleware intercepts every connection attempt, verifies the token with `jwt.verify()`, and attaches `userId` and `role` to the socket. If verification fails, the connection is rejected with `connect\_error`.

**\*\*Q9: Why is PostgreSQL the source of truth, not Socket.io?\*\***

Socket.io is a delivery mechanism — messages are not guaranteed to be received if the client is offline. All messages are first written to PostgreSQL, then the socket event is emitted. If the client is offline, they load the message history via REST on next connection.

**\*\*Q10: Why Redis for OTPs and not PostgreSQL?\*\***

PostgreSQL doesn't have native row-level TTL (automatic expiration). We would need a scheduled cleanup job. Redis TTL deletes the key atomically after the specified time — perfect for OTPs with a 5-minute validity window. OTPs also must not persist after use, making Redis the correct choice.

**\*\*Q11: How does OTP expiration work?\*\***

When the OTP is stored in Redis: `redis.set(key, value, { ex: 300 })`. This sets a 300-second TTL. After 300 seconds, Redis automatically deletes the key. Any verification attempt after expiry returns `null` from `redis.get()`, which is treated as "OTP expired."

**\*\*Q12: How is OTP brute-force prevented?\*\***

The OTP data stored in Redis includes an `attempts` counter: `{ otp, attempts }`. On each failed verification, `attempts` is incremented. When `attempts >= 5`, the OTP is deleted and the provider must ask the customer to generate a new one. Combined with the 5-minute TTL, this makes brute-forcing practically impossible on 6-digit OTPs.

**\*\*Q13: How are service proofs more trustworthy than self-reported completion?\*\***

Photos with timestamps are uploaded to Cloudinary and stored in PostgreSQL. Both customer and provider can upload before/after photos. This creates a verifiable evidence trail that can resolve disputes. Neither party can fabricate photos retroactively.

**\*\*Q14: Why not store proof images directly in PostgreSQL?\*\***

PostgreSQL is not optimized for storing large binary data (BLOBs). This would inflate database size, increase backup time, and hurt query performance. Cloudinary provides CDN delivery, image optimization, and proper binary storage. PostgreSQL only stores the metadata (URL, public\_id).

**\*\*Q15: How are reviews linked to bookings (no fake reviews)?\*\***

The Review model has `bookingId` as a `@unique` field. This means one review per booking, enforced at the database level. The controller additionally checks that: (1) The booking is COMPLETED, (2) The customer owns the booking, (3) No review exists yet for this booking. Three layers of prevention.

**\*\*Q16: How is the provider's average rating calculated?\*\***

After each new review, the backend runs `prisma.review.aggregate({ \_avg: { rating: true }, \_count: { rating: true } })` inside a transaction. This computes the true database-level average, not an incremental running average that could drift. The result is stored back on the Provider record.

**\*\*Q17: What is the trust score and how is it calculated?\*\***

The trust score is a deterministic formula (NOT AI) with four components: verification status (30pts), normalized average rating (25pts), completed jobs capped at 100 (25pts), and experience years capped at 10 (20pts). The score and all contributing factors are returned in the API response for full transparency.

**\*\*Q18: How does conversation authorization work?\*\***

Each Conversation record stores `customerId` and `providerId`. Any request to read messages or join the socket room first queries the database: `findFirst({ where: { id, OR: [{ customerId: userId }, { providerId: userId }] } })`. If the user isn't a participant, they get 404 (not 403, to avoid leaking conversation existence).

**\*\*Q19: What happens if Redis is unavailable during OTP generation?\*\***

`getRedis()` throws a structured error with `statusCode: 503`. The Express error handler returns `503 Service Unavailable`. The error is clearly logged. No silent failure occurs. All other endpoints continue working.

**\*\*Q20: What happens if a Socket.io client disconnects mid-conversation?\*\***

Socket.io has automatic reconnection configured (5 attempts, 1s delay). Messages sent while disconnected are stored in PostgreSQL and fetched via REST on reconnection. The conversation history is never lost. Only real-time delivery is interrupted.

**\*\*Q21: What data belongs in PostgreSQL vs Redis?\*\***

PostgreSQL: Users, providers, bookings, conversations, messages, reviews, proofs — anything permanent or relational. Redis: OTPs (must expire), rate limit counters (time-windowed, ephemeral). Never use Redis as a primary database; it's a cache/ephemeral store only.

**\*\*Q22: How do you ensure no credentials are leaked?\*\***

(1) All secrets are in `.env` — never in source code. (2) `.env` is in `.gitignore`. (3) Cloudinary API secret is server-side only. (4) Frontend only receives non-sensitive data (URLs, not credentials). (5) JWT payload contains only `id` and `role`, not password. (6) Password fields are never returned in API responses.

# 20-development-log

## Chapter 20: Chronological Development Log

### Phase 1 Execution Log

#### Baseline Foundation Setup

- Initialized Node.js Express ES Modules backend (`"type": "module"`).
- Set up React 18 + Vite frontend with Tailwind CSS styling.
- Created centralized Zod validation middleware and global AppError handler.

#### Neon PostgreSQL Database Integration

- Configured Prisma ORM schema (`schema.prisma`) targeting hosted **Neon PostgreSQL**.
- Synchronized `User` table model and `Role` enum via `npx prisma db push`.
- Generated `@prisma/client` JavaScript bindings.

#### Authentication & Authorization System Completed

- Created `authValidator.js` Zod schemas (`registerSchema`, `loginSchema`).
- Implemented `POST /api/v1/auth/register` with `bcryptjs` password hashing (10 salt rounds).
- Implemented `POST /api/v1/auth/login` returning JWT token (`7d` expiration).
- Built `authenticateToken` and `authorizeRoles` middlewares.
- Built `GET /api/v1/auth/me` protected user endpoint.
- Created automated Node.js test suite (`tests/auth.test.js`) verifying all 16 backend test scenarios (100% pass).

#### Frontend Authentication UI Completed

- Built `AuthContext.jsx` with real backend API integration, token persistence, and Axios header synchronization.
- Created `ProtectedRoute.jsx` and `RoleProtectedRoute.jsx`.
- Created responsive `Login.jsx` and `Register.jsx` pages with validation and error alerts.
- Built role-based dashboards (`CustomerDashboard.jsx`, `ProviderDashboard.jsx`, `AdminDashboard.jsx`).
- Added `Unauthorized.jsx` (403) and `NotFound.jsx` (404) error pages.

---

\*PHASE 1 STATUS: COMPLETE\*





























































